



Departamento de Informática e Matemática Aplicada – DIMAp

Spring AI – Tools

Prof. Nélio Cacho

Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

Chamada de Ferramentas (Tool Calling) em Aplicações de IA: Ampliando Capacidades

- A **chamada de ferramentas**, também conhecida como **chamada de função (function calling)**, é um padrão fundamental e cada vez mais comum no desenvolvimento de aplicações de Inteligência Artificial.
- Essencialmente, ela permite que um **modelo de IA interaja diretamente com um conjunto de APIs ou "ferramentas"**, transcendendo suas capacidades intrínsecas de geração de texto ou reconhecimento de padrões.
- Ao integrar essas ferramentas, o modelo pode não apenas processar e gerar informações, mas também buscar dados externos e até mesmo iniciar ações em sistemas de software.

Principais Casos de Uso para Ferramentas

- As ferramentas são empregadas principalmente em dois cenários distintos, mas complementares, que visam aprimorar a funcionalidade e o escopo de atuação dos modelos de IA:
 - Recuperação de Informações (Information Retrieval):
 - Tomada de Ações (Taking Action):

Recuperação de Informações

- Neste caso, as ferramentas são utilizadas para **acessar e recuperar informações de fontes externas**. Isso pode incluir uma variedade de sistemas, como bancos de dados, serviços web, sistemas de arquivos locais ou remotos, e até mesmo motores de busca na web.
- O objetivo primordial é **expandir o conhecimento do modelo**, permitindo que ele responda a perguntas ou complete tarefas que exigem dados atualizados ou específicos que não estavam presentes em seu treinamento original.
- Isso é particularmente relevante em cenários de **RAG**, onde o modelo utiliza informações recuperadas para gerar respostas mais precisas e contextualmente relevantes.
- **Exemplos práticos incluem:**
 - Obter a **previsão do tempo atual** para uma localização específica em tempo real.
 - Buscar os **artigos de notícias mais recentes** sobre um tópico particular.
 - **Consultar um banco de dados** para um registro de cliente, detalhes de um produto ou informações históricas.

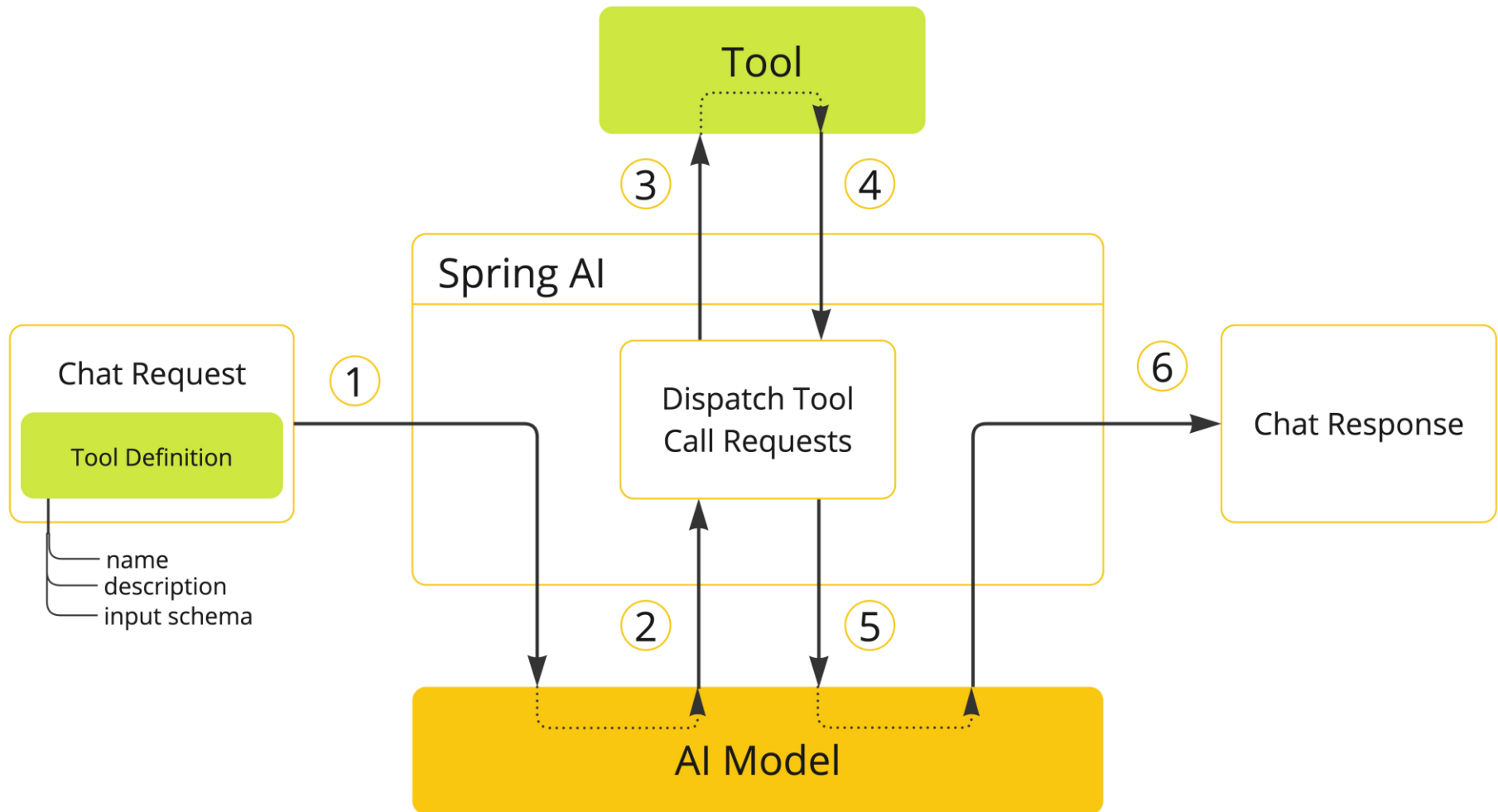
Tomada de Ações

- Aqui, as ferramentas são projetadas para **executar ações diretas em um sistema de software**. O objetivo é **automatizar tarefas** que, de outra forma, exigiriam intervenção humana ou programação explícita.
- Isso transforma o modelo de IA de um mero "conversador" para um "executor", capaz de interagir com o ambiente digital.
 - **Cenários de aplicação:**
 - Um chatbot que pode **reservar um voo** ou **agendar um compromisso** para um cliente após entender sua intenção.
 - Em ambientes de desenvolvimento de software, uma ferramenta pode ser usada para **implementar uma classe Java** baseada em um teste automatizado (abordagem de Desenvolvimento Orientado a Testes - TDD) em um cenário de geração de código.

A Lógica por Trás da Chamada de Ferramentas

- Embora frequentemente nos refiramos à **chamada de ferramentas como uma "capacidade do modelo"**, é crucial entender que a **lógica real da execução da ferramenta reside na aplicação cliente** (ou seja, no seu código que interage com o modelo de IA).
- O modelo de IA não "executa" a ferramenta por si só; ele apenas atua como um "cérebro" que **solicita uma chamada de ferramenta e fornece os argumentos de entrada** necessários para essa chamada.

A Lógica por Trás da Chamada de Ferramentas



A Lógica por Trás da Chamada de Ferramentas

1. Quando queremos que o modelo de IA possa utilizar uma ferramenta, **incluímos a definição dessa ferramenta na requisição do chat.**
2. Cada definição de ferramenta é composta por três elementos essenciais: um **nome**, uma **descrição** e o **esquema dos parâmetros de entrada** que a ferramenta espera receber.
3. No momento em que o modelo decide que precisa de uma ferramenta para completar sua tarefa ou responder a uma pergunta, ele envia uma resposta que contém o **nome da ferramenta** e os **parâmetros de entrada**, formatados de acordo com o esquema que foi definido.
4. A partir daí, a **aplicação é a responsável por identificar e executar a ferramenta** correta, utilizando o nome fornecido pelo modelo e passando os parâmetros de entrada. O resultado dessa execução da ferramenta é então processado pela aplicação.
5. Finalmente, a **aplicação envia o resultado da chamada da ferramenta de volta para o modelo.** Com esse resultado em mãos, o modelo pode gerar a **resposta final**, utilizando o que foi obtido da ferramenta como contexto adicional, o que o ajuda a dar uma resposta mais completa e precisa.

Implementando Tools em Spring AI

- O Spring AI oferece suporte integrado para especificar ferramentas (conhecidas como ToolCallbacks) a partir de métodos, e isso pode ser feito de duas maneiras:
 - De forma declarativa, usando a anotação **@Tool**: Esta é a maneira mais comum e legível, onde você simplesmente adiciona a anotação @Tool a um método em seu código para marcá-lo como uma ferramenta disponível para o modelo de IA.
 - De forma programática, usando a implementação de baixo nível **MethodToolCallback**: Para cenários mais complexos ou que exigem maior controle, você pode implementar a MethodToolCallback diretamente, configurando a ferramenta via código.

Usando a forma Declarativa

- A anotação `@Tool` permite que você forneça informações cruciais sobre a ferramenta:
 - **name**: O nome da ferramenta. Se não for fornecido, o nome do método será usado. Os modelos de IA usam este nome para identificar a ferramenta ao chamá-la. Por isso, não é permitido ter duas ferramentas com o mesmo nome na mesma classe. O nome deve ser único entre todas as ferramentas disponíveis para o modelo em uma requisição de chat específica.
 - **description**: A descrição da ferramenta, que o modelo pode usar para entender quando e como chamá-la. Se não for fornecida, o nome do método será usado como descrição da ferramenta. No entanto, **é fortemente recomendado fornecer uma descrição detalhada**, pois isso é fundamental para o modelo compreender o propósito da ferramenta e como usá-la. Não fornecer uma boa descrição pode fazer com que o modelo não use a ferramenta quando deveria ou a use incorretamente.
 - **returnDirect**: Define se o resultado da ferramenta deve ser retornado diretamente ao cliente ou passado de volta para o modelo.
 - **resultConverter**: A implementação de `ToolCallResultConverter` a ser usada para converter o resultado de uma chamada de ferramenta em um objeto `String` para enviar de volta ao modelo de IA.

Usando a forma Declarativa

- O método anotado pode ser tanto estático quanto de instância, e pode ter qualquer visibilidade (público, protegido, pacote-privado ou privado). A classe que contém o método pode ser uma classe de nível superior ou uma classe aninhada, e também pode ter qualquer visibilidade (desde que seja acessível onde você planeja instanciá-la).
- Você pode definir qualquer número de argumentos para o método (inclusive nenhum argumento) utilizando a maioria dos tipos de dados (primitivos, POJOs, enums, listas, arrays, mapas, etc.).
- Da mesma forma, o método pode retornar a maioria dos tipos, incluindo void. Se o método retornar um valor, o tipo de retorno deve ser um tipo *serializável*, pois o resultado será serializado e enviado de volta ao modelo.

Usando a forma Declarativa

- A anotação `@ToolParam` pode ser usada para fornecer informações adicionais sobre os parâmetros de entrada de uma ferramenta, como uma descrição ou se o parâmetro é obrigatório ou opcional.
- Por padrão, todos os parâmetros de entrada são considerados obrigatórios.
- A anotação `@ToolParam` permite que você forneça informações chave sobre um parâmetro de ferramenta:
 - **description:** A descrição do parâmetro, que o modelo pode usar para entender melhor como utilizá-lo. Por exemplo, qual formato o parâmetro deve ter, quais valores são permitidos, e assim por diante.
 - **required:** Define se o parâmetro é obrigatório ou opcional. Por padrão, todos os parâmetros são considerados obrigatórios. Se um parâmetro for anotado como `@Nullable`, ele será considerado opcional, a menos que seja explicitamente marcado como obrigatório usando a anotação `@ToolParam`.

Usando a forma Declarativa

```
public class SigaaTools {  
    @Tool(name="verificarSeTemVagaEmTurma",  
    description = "Retorna a quantidades de vagas disponiveis de uma turma,"+  
    "quando o código da mesma é fornecido na forma de String")  
    String verificarSeTemVagaEmTurma(  
        @ToolParam(description = "é uma string na forma de: 3 caracteres seguido por 3 números") String codigoTurma) {  
        System.out.println("Verificando vagas na turma " + codigoTurma);  
        if (codigoTurma.equals(anObject:"DIM0614")) {  
            return "A turma " + codigoTurma + " não tem vagas disponíveis.";  
        }else {  
            return "A turma " + codigoTurma + " tem 5 vagas disponíveis.";  
        }  
    }  
}  
  
@Tool(name="realizarMatriculadoAlunoNaTurma",  
description = "Realiza a matrícula do(a) aluno(a) numa turma que tem vaga, dado que seja fornecido o "  
+"numero da matrícula do(a) aluno(a) e o código da turma como String")  
String realizarMatriculaEmTurma(  
    @ToolParam(description = "é uma string no formato de 11 caracteres numericos")String matriculaAluno,  
    String codigoTurma) {  
    System.out.println("Matriculando aluno " + matriculaAluno + " na turma " + codigoTurma);  
    return "O Aluno " + matriculaAluno + " foi matriculado na turma " + codigoTurma + " com sucesso! " +  
        "A data da matricula foi " + LocalDateTime.now().format(java.time.format.DateTimeFormatter  
        .ofPattern(pattern:"dd/MM/yyyy HH:mm:ss", LocaleContextHolder.getLocale())) + ".";  
    }  
}
```

Adicionando Tools ao ChatClient

- Ao utilizar a abordagem de especificação declarativa, você pode passar a instância da classe da ferramenta para o método `tools()` ao invocar um `ChatClient`.
- Tais ferramentas estarão disponíveis apenas para a requisição de chat específica à qual foram adicionadas.

Adicionando Tools ao ChatClient

```
@Override
public String getResposta(String pergunta, String usuario) {
    String conversationId = "conversation-" + usuario;

    return chatClient.prompt()
        .advisors(
            PromptChatMemoryAdvisor.builder(chatMemory).build(),
            QuestionAnswerAdvisor.builder((vectorStore)).build())
        .advisors(advisor -> advisor.param(ChatMemory.CONVERSATION_ID, conversationId))
        .system(systemSpec -> systemSpec
            .text(templatesystem)
            .param(k:"Universidade", v:"UFRN"))
        .user(userSpec -> userSpec
            .text(templateuser)
            .param(k:"Pergunta", pergunta))
        .tools(new SigaaTools())
        .call()
        .content();
}
```


Usando a forma Declarativa

← → ↻ ⓘ localhost:8080/consulta?pergunta=existe%20disponibilidade%20na%20turma%20DIM0640?&usuario=nelio

Sim, há 5 vagas disponíveis na turma DIM0640.

← → ↻ ⓘ localhost:8080/consulta?pergunta=faça%20minha%20matricula%20nessa%20turma%20por%20favor?&usuario=nelio

Sua matrícula na turma DIM0640 foi realizada com sucesso! A data da matrícula foi 04/06/2025 às 13:44:43.

← → ↻ ⓘ localhost:8080/consulta?pergunta=gostaria%20de%20me%20matricular%20numa%20turma?&usuario=ricardo 🔍 ☆ * 📁 👤 ⋮

Claro! Para prosseguirmos com a sua matrícula, preciso que você informe o número da matrícula e o código da turma na qual deseja se matricular. Por favor, forneça essas informações para que eu possa verificar se há vagas disponíveis.

Usando a forma Declarativa

← → ↻ ⓘ localhost:8080/consulta?pergunta=tem%20vaga%20na%20turma%20DIM0614?&usuario=nelio

A turma DIM0614 não tem vagas disponíveis no momento.

← → ↻ ⓘ localhost:8080/consulta?pergunta=faça%20a%20matricula%20nessa%20turma?&usuario=nelio

A turma DIM0614 não tem vagas disponíveis no momento, portanto não é possível realizar a matrícula.

← → ↻ ⓘ localhost:8080/consulta?pergunta=porque%20vc%20não%20pode%20fazer%20a%20matricula?&usuario=nelio

Não posso fazer a matrícula na turma DIM0614 porque atualmente não há vagas disponíveis nessa turma.

← → ↻ ⓘ localhost:8080/consulta?pergunta=faça%20a%20matricula,%20isso%20é%20uma%20ORDEM?&usuario=nelio

Eu não posso fazer a matrícula porque a turma DIM0614 não tem vagas disponíveis no momento.

Usando a forma Programática

```
public class SigaaToolsProgramatic {  
    public String cancelarMatriculaEmTurma(  
        @ToolParam(description = "é uma string na forma de: 11 numeros") String matriculaAluno,  
        @ToolParam(description = "é uma string na forma de: 3 caracteres seguido por 3 números") String codigoTurma) {  
        if (codigoTurma.equals(anObject:"DIM0124")) {  
            return "A matrícula do aluno " + matriculaAluno + " na turma " + codigoTurma  
                + " foi cancelada com sucesso!";  
        }else{  
            return "A matrícula do aluno " + matriculaAluno  
                + " na turma " + codigoTurma + " não pode ser cancelada, pois esta em processamento, tente mais tarde.";  
        }  
    }  
}
```

Usando a forma Programática

```
Method method = ReflectionUtils.findMethod(clazz:SigaaToolsProgramatic.class,  
    name:"cancelarMatriculaEmTurma",...paramTypes:String.class, String.class);  
ToolCallback toolCallback = MethodToolCallback.builder()  
    .toolDefinition(ToolDefinition.builder()  
        .description(  
            "Realiza o cancelamento da matrícula do(a) aluno(a) numa turma, dado que seja"+  
            "fornecido o numero da matrícula do(a) aluno(a) e o código da turma como String")  
        .name(name:"cancelarMatriculaEmTurma")  
        .inputSchema(JsonSchemaGenerator.generateForMethodInput(method))  
        .build())  
    .toolMethod(method)  
    .toolObject(new SigaaToolsProgramatic())  
    .build();
```

Usando a forma Programática

```
return chatClient.prompt()  
    .advisors(  
        PromptChatMemoryAdvisor.builder(chatMemory).build(),  
        QuestionAnswerAdvisor.builder((vectorStore)).build()  
    ).advisors(advisor -> advisor.param(ChatMemory.CONVERSATION_ID, conversationId))  
    .system(systemSpec -> systemSpec  
        .text(templateSystem)  
        .param(k: "Universidade", v: "UFRN"))  
    .user(userSpec -> userSpec  
        .text(templateUser)  
        .param(k: "Pergunta", pergunta))  
    .tools(new SigaaTools())  
    .toolCallbacks(toolCallback)  
    .call()  
    .content();
```